

DSA STUDY BLUEPRINT SERIES

44. Merge Sort Algorithm

Divide-and-Conquer Recursive Merge Sort

Section 06 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Concept:** Divide array, recursively sort halves, then merge sorted lists.
- **Stability:** Stable sorting algorithm that preserves original relative order.

Memory & Execution Blueprint

[4, 2, 1, 3]

→ Divide →

[4, 2] • [1, 3]

C++ Implementation Reference

Standard code layout and syntax

```
void mergeSort(int arr[], int l, int r) {  
    if (l >= r) return;  
    int mid = l + (r - l) / 2;  
    mergeSort(arr, l, mid);  
    mergeSort(arr, mid + 1, r);  
    merge(arr, l, mid, r);  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(N \log N)$ (All cases)
Space Complexity	$O(N)$ (Auxiliary array)

Key Practices & Gotchas

- **Important:** Ideal for sorting linked lists due to sequential memory access.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's dry run the partition splitting sequence for Merge Sort on list `[4, 2, 1, 3]` :

Depth Level	Divided segments	Sorting result	Merged result
1	[4, 2] & [1, 3]	Recursive sorting	[2, 4] & [1, 3] merged to [1, 2, 3, 4]

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Pivot heuristics:** Quick Sort speeds depend on pivot balance; choosing middle or random elements prevents $O(N^2)$ degradation.
- **Inversions:** Count array inversions during merge sorts to identify out-of-order bounds sizes.
- **Related LeetCode Problems:** #912 Sort an Array, #315 Count of Smaller Numbers After Self.